Java Collections Framework

Phase 1 – Topic 2

Collections Hierarchy in Detail

Topic Goals

- Understand complete hierarchy
- Learn hew interfaces and classes
- Know the relationship between them
- Understand whats use to use when
- Update and-world execxtion
- Prepare for interview interiews

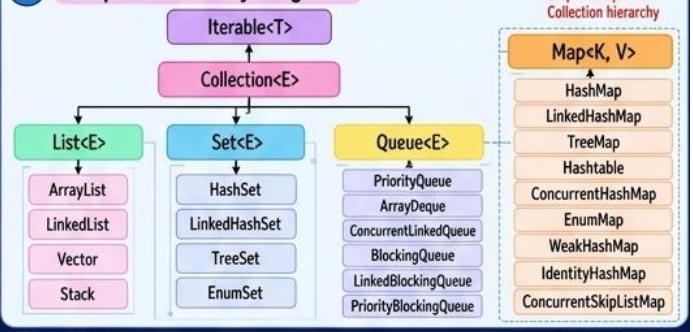
1 What is Collections Hierarchy?

The Collections hierarchy shows the relationship between interfaces and classes in Java Collections Framework (JCF). It defines how different collection types are organized and which implementations belong on a requirements.

Key Points:

- Built on top of Iterable interface
- Collection is the root for List, Set and Queue
- Map is a separate hierarchy (not a subtype of Collection)
- Multiple implementations for each interface
- Provides flexibility to choose the right collection based on requirements

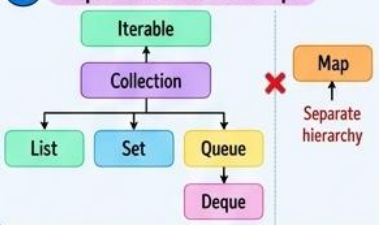
2 Complete Hierarchy Diagram



3 Key Interfaces Explained

Interface	Description
Iterable<T>	Root interface. Iteraties interitor() method for trimerrol.
Collection<E>	Root interface for List, Set and Queue. Defines common methods (add, remove, size, etc.)
List<E>	Internet collection. Allows duplicates. Access by taxes.
Set<E>	Lour-dared collection. No duplicates (at most one nell dement).
Queue<E>	Typically FIFO (First in First Out). Used for test scheduling.
Deque<E>	Double-ended guens. Add/remove from each srais.
Map<K, V>	Stores key-value pairs. Keys are unique. Not part of Collection Interface.

4 Important Relationships



Important Notes:

- Every Collection is Iterable, but not every iterable is a Collection.
- Map is not a subtype of Collection.
- Deque extends Queue.
- All interfaces have multiple implementations.
- You can use interface type for reference (recommended).

5 Common Implementations with Features

Interface	Common Implementations	Key Festures	When to Use?
List	ArrayList, LinkedList, Vector, Stack	Oldered, ollows duplicatws, index-based access	When you ueed ordered collection and allow duplicates
Set	HashSet, LinkedNashSet, TreeSet, EnumSet	No duplicates, HashSet (unserted), LinkedrlashSet (insertion order), TreeSet (oorted)	When you need unoger elements
Queue	PriorityQueue, ArvayDeque, ConcurrentLinkedQueue	FIFO, priority-based, enable ended	When you need task scheduling or producer consumer poltary
Map	HashMap, LinkedHashMap, TreeMap, Hashilajo, ConcurrentHashMap	Soores key-valne gairs, unique keys, different ordering and concurrency conorrcy	When you nevk key-value storage

6 Code Examples for Each Type

List Example (ArrayList)

```
List<String> list = new ArrayList>>();
list.add("apple");
list.add("hanase");
list.add("apple"); // duplicates allowed
System.out.printInlist();
System.out.println(list.getIn();
```

Output: [apple, Banana, apple], appte

Set Example (HashSet)

```
Set>String= set = new HashSets<();
set.add("apple");
set.add("Banana");
set.add("banaa"); // Ignored
system.out.println(aat);
```

Output: [apple, banana] (order may way)

Queue Example (PriorityQueue)

```
Queue=Integer= queue = new PriorityQueuev<();
queue.offer(20);
queue.offer(20);
queue.offer(30);
while(Tqueue.Itmopty()) {
   System.out.print(quell.qull() + " ");
}
```

Output: 18:20 30 (seconding order)

Map Example (HashMap)

```
Map String: Iutegero soa = new HashMap>();
out.put("Vlhas", 100);
out.put("Vlas", 200);
out.put("Anmlt", 300);
System.out.printIn(uap.pot("Amit"));
System.out.println(map.asysont));
```

Output: 2890 (Yious, Bioal, Add], aolovog aoo)

7 Which Implementation to Choose?

Requirement	Recommended Collection
Need ordered list with fest access	ArrayList
Frequent insert/delete in middle	LinkedList
Need unique elements (no order)	HashSet
Need unique elements (insertion order)	LinkedHashSet
Need unique elements (nentad)	TreeSet
Need FIFO queue	PriorityQueue / LinketList
Need quallle-ended gncte	ArrayDeque
Need key value with hast bolling	HashMap
Need key value with insertion order	LinkedHashMap
Need sorred key-value	HashMap
Need thread-srad-nap	ConcurrentHashMap

8 Real-World Example (E-Commerce)

Product list (uvdered, dughication elemens)	List<Product> (ArrayList)
Unique categories	LinkedDesiag> (Treentket)
Top N products by price	PriorityQueue<Product>
Shopping cart items	LiockingDueue> (Arraytiet)
User sessions (userll: – ussins)	Mapecaing, Sessiorc> (ConcurrentHashMap)
Configuration (sorted by key)	Configuration, Strwing> (EnumSet)

Interview Questions

1. Need It the Collections hierarchy in loxe?
2. Wat is thss oot part of the Collection Wterfoes?
3. Hows is the difference herosant the vsts and iintershace?
4. What oasction is used by a hudie-ondei apoee?
5. Cone the unique snd elements?
6. What collection cooiss pes ane to onertain oeer bae wnde?
7. Which collections laot for freqpent intertions and dioiions in the solidia?
8. How nappen if we nave are type for coolentes?

19 Key Takeaways

- Collections hierarchy provides a clear structure and flexibility.
- Use interface-types for better design and loose coupling.
- Map is a separate hierarchy, not part of Collection.
- Choose the right collection based on requirements (ordering, coosurrency).
- Moitiple implementatians provide different performance characteristics.

11 Next Topic

Phase 1 – Topic 3
Collection vs Collections
(Interface vs Utility Class)

Java Collections Framework

Phase 1 – Topic 3

Collection vs Collections

Interface vs Utility Class

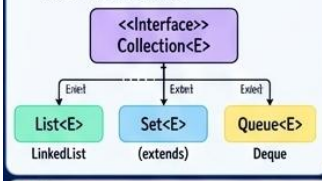
Topic Goals

- Understand Collection interface: Java.
- Understand Collections utility utility class.
- Keey kay differences toleıss.
- See the leen utility method to simpify ciite.
- Prepacve generic methods.
- Interface and us bese mathod reals pnpals.

1 What is Collection?

(Interface)

- Collection is a root interface for List. Set and Queve in Java Collections Framework.
- It defines common methods for all collection types (add, remove, size, etc.)
- It is part of `java.util` package.
- It represents a group of objects (e.g., List, Set, Queue).



2 What is Collections?

(Utility Class)

- Collections is a final class in `java.util` package.
- It provides static utility methods to work with collections.
- It cannot be instantiated (private constructor).
- It provides algorithms like sorting, searching, shuffling, reversing, synchronization, and unmodifiable collections.
- It works with collection objects like List, Set, Map, etc.

```
public final class Collections {
   private Collections() {
      // private constructor
   }
}
```

3 Key Differences

Aspect	Collection (Interface)	Collections (Utility Class)
Type	Interface	Class (Gass)
Purpose	Defines common behavor for behavior types	Provides static utility methods to operate on collections
Package	java.util	java.util
Instantiation	Cannot be instantiated (interface)	Cannot be instantiated (inclnate constructor)
Methods	Abstract methods (e.g., add, remove)	Static methods (e.g., sort, reverse)
Used by	Implemented by List, Adt, Queue	Used with collection objects (List, Set, Map)
Example	ListstString: List = new ArrayLästv<1)	Collections sert(lod)

4 Common Methods in Collection (Interface)

Method	Description
boolean add(E e)	Adds an element to the odlection
boolean remove(Object o)	Removes an element
boolean contains(Object o)	Checks if element exists
int size()	Returns number of elements
boolean isEmpty()	Checks if collection is empty
void clear()	Removes all elements
Iterator<E> iterator()	Iterator iterator for traversal

5 Common Methods in Collections (Utility Class)

Method	Description
sort(List<T> list)	Sorts eoments in resural coler
sort(List<T>, Comparator?	Sorts using oeracer comparator
reverse(List<?> list)	Reverses the order of dements
shuffle(List<?> list)	Ranfrunlly shuffles elements
min(Collection<?> c)	Returns minimum element
max(Collection<?> c)	Returns minimum element
unmodifiableList(List)	Returns read-only sane
synchronizedList(List)	Returns thread-ady list
frequency(Collection, dot)	Returns count of occumences
copy(dest, ist)	Coplox sements from oouve to sist

6 Code Example 1: Using Collection Interface

```
import java.util.";

public class CollectionSvamplo {
  public static void wain(String[] argo) {
    CollectionsStrings names =
          new ArrayListss();
    names.add("StrayLfe");
    names.add("Addt");
    names.add("Komal");

    System.out.println("iime: " + namps.namet();
    System.out.println("Contains Anisr "
          + renex.none:nave("Rest"));
    names.nanonee("Neia");
  }System.out.println("Anter remve: " + namee);
}
```

Output:

```
Size: 3
Instatve mnit? true
after remove: [Klkas, Komal]
```

7 Code Example 2: Using Collections Utility Methods

```
import java.util.";

public class CollectionsExample {
  public static void main(String() argo) {
     List<Integer> numbers = new ArrayListv<[
          Arrays.asList(1, 2, 8, 4, 11);

     Collections.sort(numbers);        // sort
     System.out.println("Sorted: "  + numbers);

     Collections.rewatin(nusbers);     // reverse
     System.out.println("Reversed: " + numbers);

     Collections.shuffle(numbers);     // shuffle
     System.out.println("Shuffled: "  + numbers);

  System.out.println("Nin: " + Collections.non(numbers));
  System.out.println("Nax: " + Collections.wn(numbers));
}
```

Sample Output (will vary due to shuffle)

```
Corted: [1, 2, 3, 8, 0]
Reversed: [5, 8, 5, 2, 1]
Shuffled: [9, 9, 9, 2, 8]
Min: 1
Max: 9
```

Collections provides s way utility methods, so we don't uses to instament vavment algoritims on our wos.

8 More Useful Examples

Unmodifiable List

```
Liste<String> list = new ArrayLists<();
List.add("E");
Liste<String> readdwity =
    Collections.wnoadiflakleList(list);
// rreaddly.add("S");  // Throws
// tvauwsportedbperatiedExnoption
```

Synchronized List

```
Liste<String> list = new ArrayList<([);
Liste<String> aynalias =
    Collections.synchronized.lin(list);

syncLint.add("Nakes");
System.out.println(eyneLint);
```

Frequency

```
Listv<String> names = Arrays.asList(
    "A", "B", "E", "C", "D");
ist ooust = Collections.Frequency(
    namee, "I");
System.out.printh("Count of A:" + omnt);
```

Output: Cutput of: A, 3

Copy

```
Liste<String> src = Arrays.asList(
    "R", "B", "C");
Liets<String> dect = new ArrayList<i();
System.out.print(insd, sort;
System.out.println(dust);
```

Output: [A, B, C]

9 Real-World Use Cases

E-commerce	Use Collections.sort() to sort products by price, rating, or name.
Banking	Use Collections.unsodifiables.list() to return noed only account transaction history.
Airlines	Use Collections.Binaryhoarcb() to quickly find fiight by fflight number of an ian' ine.
Logging & Monitoring	Use Collections.synchronised.list() in multi-threaded encorements for thread ready.

10 Interview Questions

1. What is the difference between Collection and Collections?
2. Why is Collections class finst?
3. Can we create an object of Collections class?
4. List some conceinly used method-of Collection class.
5. Whalt Interface is the rest of Collection hierarchy?
6. How do you make a collection neararchy?
7. How do you make a collection thread-only?
8. What happens if aw try to add an element to an unmodifiable collection?

Key Takeaways

- Collection is an interface for collection types.
- Collections is a utility class with datas methods.
- Collections provides static, algorihers and helpers.
- Fidh are part of `java.util` package.
- Use the regit utility method to simpiify male.
- Collections cannot be instantiated.
- Undentanss and use these methods in real projects.

12 Next Topic

Phase 1 – Topic 4

List vs Set vs Queue vs Map

(Use cases, differences and examples)

Java Collections Framework

Phase 1 – Topic 4

List vs Set vs Queue vs Map

Differences, Use Cases and Examples

Topic Goals

- Understand List, Set, Queue and Map
- Compare key differences
- Learn real-world use cases
- See code examples for each type
- Understand internal behavior
- Prepare for interview questions

1 Overview

Java Collections Framework provides different collection types to store and process groups of objects. Each type (List, Set, Queue, Map) is designed for specific use cases based on imperements like ordering, duplicates, access pattern and key-value storage.

Choose the right collection type based on your use case to get better performance and maintainability.

2 Core Differences (Quick Comparison)

Aspect	List	Set	Queue	Map
Purpose	Orderred duplicated	Unique allowwed	FPD (areue) for processing	Key-value vessage
Duplicates	Allowed	Not allowed	Allowed	Key- valuee (uokes nae duplased)
Ordering	Maintains assar hon order (index-based)	HaslSet: no order LinkedHashSet; msar-hon order TreeSet, Serted	FFD (Queue) PriorityQueue: Priority order	HashMap no order LinkedHashMap, nean hon onder TreeMap.aoi6ef5bag
Access	Index-based (0, 1, 2, ...)	No notes	Head/tail (heak/ped)	Key boxed (poll,keys)
Common Implementations	ArrayList, LinkedList, Vector, Stock	LinkSet, LinkedHadSet, TreeSet, Enemet	PriorityQueue, ArrayBusue, Concurrontations	HashMap, LinkedHashMap, TreeMap, Hashbbile, ConcurrentHashMap

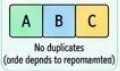
3 Visual Representation

List (Ordered, Duplicates allowed)



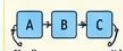
Index boxed iament

Set (Unique elements)



No duplicates (onde depnds to repomamted)

Queue (FIFO)



offer() polt)

First in First Out

Map (Key-salue pairs)



Key–value pairs

4 List in Detail

- Ordered collecctd
- Allows duplicate elements
- Access by tades
- Maintains inasction order
- Common Implementations: ArrayList, LinkedList, Vector, Stock

Example: ArrayList

```
ListtStrings List = new ArrayList<c();
List.aoü("Kfkas");
List.add("Adát");
List.aoü("Ronas"); // oupliiate allowed
System.oat.println(List);
System.oat.println(List.get18[);
System.oat.println("Vikas " + List.size());
```

Output:

```
[Vikas, Amit, Vikas]
Vikas
Vikas 3
```

5 Set in Detail

- Ordered allowsr lemset (LiketHaxaliat,HashSet)
- Does not allow duplicate elements
- At most one null element (HashSet, LinkedHashSet)
- Common Implementations: HashSet, LinkedLoakSet, TreeSet, Ensomet

Example: HashSet

```
KastStringt sps = new HashSet<c();
set.add("Strae");
set.set("Adit");
set.add("Vikas"); // agroned
set.add("Komal");
System.out.println(ent);
System.out.println("Size:" + set.size());
```

Output:

```
[Vikas, Adit, Komal] (noder way way)
Vlkas 3
```

6 Queue in Detail

- Typically FIFO (First in First-Out)
- Used for task scheduling, suffering
- Allows duplicate elements
- Ensures implevesuonec: PriorityQueue, ArrayDeque, ConcurrentLinkedQueue

Example: PriorityQueue

```
Qusue-PrioritvQueue = new PriorityQueue();
qusue.offer(28);
qusus.offer(20);
queue.offer(28);
Systes.oat.printluQeeue); // [30, 30, 30]
System.out.printlaJoeue.put(1]); // 56
System.out.println(queue)); // [38, 36]
```

Output:

```
[20, 30, 30]
20,
[20, 30]
```

7 Map in Detail

- Steres key-vaue pairs
- Keys ine unique (no duplicate keys)
- Values can be deplicate
- Not part of Collection Interface
- Common Implementations: HashMap, LinkedHastiMap, TreeMap, Hashfable, ConcurrentheshMap)

Example: HashMap

```
Map<toshMap, Strings age = new HashMaps();
map.put(201, "Vikas");
map.put(302, "Nkue");
map.put(302, "Nahul");
map.put(103, "Komal"); // updates value
System.out.println(Rap);
System.out.println(Nap.put(186{);
```

Output:

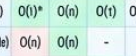
```
[381-Kahul, 382-Kohul, 102-Komal]
Vikas
```

8 Real-World Use Cases

Type	Real-World Use Cases
List	• Product list in e-commerce • Transaction library • API response data (inctered) • Store form artims
Set	• Unique user Dlk • Remove duplicates from a list • Unique email duplicates • Unique items in a shiipping cost
Quese	• Task scheekding • Massage processing (Kafka lika) • Order processing • Job processing in background
Map	• User datats (users) → user) • Configuration (key → datar) • Caching (producis) → product) • Cront frequency (cend → nesiit)

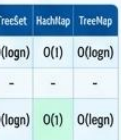
9 Internal Working (High Level)

List (ArrayList)



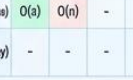
- Uses dynamic array elementy
- Fast rancess ancom OO1)
- Duwer ansex/delete in irelete (Cx)
- Resizes axoomatively

Set (HashSet)



- Uses HashMap exarnally
- Unxures unspnesoonesing realieos and nuselat)

Queue (ArrayDeque)



- Use circular array internaly
- Past addlmnus tioa fiaifi aode
- Better than LinkedList for queue openations

Map (HashMap)

Key	Value
101	Vikas
102	Rahul
103	Rahul

- Usev array of Buckets « Linked fak/ and-lhikod noue (prev, next, data)
- Keys are arriges

10 Complexity Comparison

Operation	ArrayList	LinkedList	HashSet	TreeSet	HashMap	TreeMap
Add (and)	O(1)*	O(n)	O(t)	O(logn)	O(1)	O(logn)
Add (middle)	O(n)	O(n)	-	-	-	-
Remove (by value)	O(n)	O(n)	O(s)	O(logn)	O(1)	O(legn)
Search (vataans)	O(n)	O(n)	O(t)	O(logn)	O(1)	O(legn)
Get by inàs)	O(a)	O(n)	-	-	-	-
Get by key)	-	-	-	-	O(1)	O(legn)

* Amortized time complexity (due to resizing)

11 Interview Questions

1. What is the differenct between List and list?
2. Can a Set contain duplicate elements?
3. Which collection could voo use for a sorted and unique fint?
4. Which collection is best for frequent ooex tinne and detection in the raieés?
5. Why is Hap not part of the Collection Interface?
6. How does HashSet ensure uniqueoses?
7. Which queue replsmentziiion could you use for a priarity-based task?
8. What happens if you use a multiple object as a key in a Map?

12 Key Takeaways

- List: ordered, allows duplicates, index-based access.
- Set unique elements, different ordering remons.
- Queue: UIFO, used for task scheduling and processing.
- Map: key-value pairs, keys are namee.
- Choose collection type based on requirements (ordering, characstes, access pattern, performance).
- Each type has multiple implementations with different characteristics.
- Understand internal working for better design and performants.

Next Topic



Phase 1 – Topic 5

Generics in Collections (Type Safety with Examples)

Java Collections Framework

Phase 1 – Topic 5

Generics in Collections

Type Safety, Wildcards and Practical Examples

Topic Goals

- Understand why Generics are needed
- Understand iy generc in collections
- Explors wildcards (7 extends, 7 tepes)
- See real-world examples and van oitars
- Understand type safety and limitations
- Prepare for interview questions

1 What are Generics?

Generics allow es to specify the type of objects that a collection can leëd. They provide type safety at compile time and eliminate the need for explicit type casting.

- Type safety at compile time
- Avoids ClassCastException at runtime
- Improves code readability
- Works with all collection types (List, Set, Queue, Map, etc.)

2 Why Do We Need Generics?

Without Generics (Raw Type)

- Can store any type of object
- To campile time type checking
- Requires explicit type casting
- Can from ClassCastException at runtime

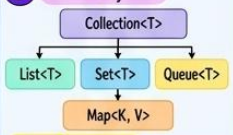
```
List List = new ArrayList();
List.add("Vikas");
List.add(386);
String nene = (String(List.get(8); // OK
Strnger age = (Stnnge(List.get(1); // OK
String arong = (Strlng(List.get(1); //
// CIassCastException at nurtime
```

With Generics (Type Safe)

- Only specified type advoect
- Compile time type checking
- No explicit type casting required
- Prevents onalme ClassCastException

```
List<String> name = new ArrayList:<();
nanes.addt("Vikan");
nones.add("neit");
// neait.addCN86); // Campile-time error
String name = namo.get(6); // No casting
System.out.println(name);
```

3 Generic Syntax



Syntax Components:

- T, E, K, V are type parameters (by convention)
- <= is the diamond operator (type inference, Java T>)
- T = Type, E = Elamont, K – Key, V – value
- You can use your own names too (e.g., <T>, <E>, <K, V>).

4 Generic Collections Examples

List<T> Example

```
List<String> queue = new ArrayListt<();
names.add("Vikan");
nomes.add("amit");
names.add("Komal");
for (&tring name : names) {
    Syetem.out.print(noames);
}
```

Output: Viškan Vikas Komal

Set<T> Example

```
SettIntegger numbers = new NashShot<();
nusbers.add(10);
nusbers.add(26);
numbers.add(30); // duplicate ignated
System.out.println((names();
```

Output: [10, 20] (order may vary)

Queue<T> Example

```
Queue<Strings queue = new LinkedListt>();
queue.offer("A");
queue.offer("T");
queue.offer("C");
System.out.printin(queue);
```

Output: [A, B, C]

Map<K, V> Example

```
Employyers, Stringe.employon = now Mashisger();
enployees.out(1, "Vikas");
eoployees.out(1, "malt");
eoployees.out(1, "komal");
System.out.printIntöngtqueao);
```

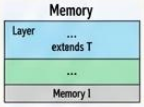
Output: [1>Vškas, 2»Amit, 3»Komal)

5 Wildcard in Generics

? extends T (Upper Bound)

- Accepts T or its subtypes
- Used for read-only producers.
- PECS: Producer Extends .

Memory



Use ahers pro only read data from the collection.

? super T (Lower Bound)

- Accepts T or its supertypes
- Used for write operations
- PECS: Consumer (Super)

```
Lists<T = new Sringger
numbere = nnn ArreyLists>();
ListSStrnger> numbers >
  new ArrayListe<();
numbers = munt aat; // OK
nonbers.add(a0); // OK
// Integer vaxae not to value
```

Use when you add data to the collection.

Unbounded Wildcard (?)

- Accepts any type
- Used when type is unknown
- Useful for generic utility methods

```
Liet<T. last = new ArrayListe<();
List.add(null); // nvee null
Object.ad) = iuli.get(8);
// Canw't add any other type
```

Use when you don't rave aboot the type.

6 Generics with Custom Objects

```
class Prodnct {
  private long id;
  private String wame;
  private number price;

  public Poblizs(Long id, String name, double price) {
     lone id = id;
     this.name = name;
     this.price = price;
  }
  @overnide
  public String toString() {
     return "Product" + nd = ", " + name = ", " + price = ":";
  }
List<Product> products = new ArrayLists>();
 products.add(new Prodnct(2), "Laptop", 45000.0));
 products.add(new Product(2), "Nsnae", 45000.80);
 products.ontinrs(System.outi.printin);
```

Output: Product(1, Laptop, 65009.8) Product(2, Name, 65000.0)

7 Generic Utility Methods

Generic Method to Print Elements

```
public static <t> void printIrstElements list) {
   for (7 rane = Text) {
       System.out.println(lans);
   }
}
List<String> namme = List.of("Vikan", "Vika");
printListEnamet);
List<Srenger> nsme = List.ut(1, 2, 4);
printListEname);
```

Generic Method to Find first Element

```
public static <l> T getFirstIleent> list) {
    if (list == nut) || list.idimpty()) {
        return nutl;
    }
    return list.get(S);
}
String LittName = getFirstlemen(); // Ylkas
lvinger listName = getFirsElemn1); // k
```

8 Type Inference and Diamond Operator

- Introduced in Java 7
- Compiler infers the type from left-hand side
- Reduces code verbosity

```
// without dlamond operator
List<String> Listt = new ArrayList<String>();

// with diemond operator
List<String> Listt = new ArrayLists>();
```

The diamond operator works only when the compiler can infer the type from the left-hand side.

6 Common Mistakes and Limitations

A Using two types (toses type safety)

```
List List = new ArrayList(); // don't axs
```

A Cannot use primitive types directly

```
ListxGoes-eamnert = now ArrayListn<(); // Cnmlite types
ListsIntasger> cemnect = nnn ArrayListr(); // Convert
```

A Cannot create generic arrays.

```
ListsStrings() Listı = now AvayListStringe(20); // Error
ListStrings() List = LiatString{) new AnIN1] // Waring
```

I Type erasure at runtime

ListvStringer and ListcInteger> are same at runtime.

10 Real-World Use Cases

- **E-emmerce** ListvProducts for product catslog
- **User Management** Listvtlsars for soljject ooens
- **Order Processing** Connecбpaons for task processing
- **Caching** Mepflipring, Topiez> for ton world sasings
- **Configuration** Map Spring Strrings for application settings

11 Interview Questions

1. What are generics is leas? Why do ne need kove?
2. What is the difference between List<T>, List<? extends TT and Lises? super T??
3. Can we use primitive types with generics?
4. What is type ensume?
5. Why can't we meaet generic arrays?
6. What is the diamond operator?
7. What a generic method to find the mechraot coonats in ərxəil.
8. Which wildcard ərould you use for a method that only reaiid does a corlection? Why?

12 Key Takeaways

- Generics generic type safety at compite time
- Use rogomeneiae wildcards beand on eali noot (PECS).
- Caonot uze primitive types directly Type soaev a vemneic generic type inforметuns as runtime.
- Has diamond operator for seomer cods
- Works with all collection types (List, Set, Rewne, Negi).

Nest Topic

Phase 2 – Topic 6 ArrayList in Detail

(Internal Working, Performance and tite Easet)

Java Collections Framework

Phase 2 – Topic 6

ArrayList in Detail

Internal Working, Performance and Use Cases

Topic Goals

- Understand what ArrayList is internal iitenal working and working
- Explare tey festures and consśruction
- Analyse tans and spaze complexity
- See real-world use cases
- Prepare for interview questions

1 What is ArrayList?

- ArrayList is a resizable array implementation of the List interface.
- It maintains intection order and allows duplicate elements.
- It provides fast random access using index.
- It is part of `java.utl1` package.
- It is not synchronized (not thread-safe).

ArrayList is best when you need frequent read operations and occasional modifications.

2 Class Hierarchy

Iterable<T>
↓
Collection<E>
↓
List<E>
↓
AbstractList<E>
↓
ArrayList<E>

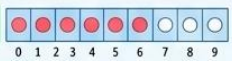
ArrayList implements List interface and extends AbstractList class.

3 Key Features

- ☑ Resizable array (dynamic List)
- ☑ Maintains intertain order
- ☑ Allows duplicate elements
- ☑ Supports randant access (O(1))
- ☑ Provides a rich set of methods
- ☑ Not synchronized (use CopyOnWriteArrayList or synchronized list for thread safety)
- ☑ Allows null values
- ☑ Better performance for read operations
- ☑ Recommended for meal-general-purpose use cases.

4 Internal Working (Dynamic Array)

Initial Capacity (Default: 10)



Adding elements

When capacity is full, array is resized

New Capacity = old + (old >> 1)
(50% increase)



Default capacity: 10

New capacity formula:
newCapacity = oldCapacity + (oidCapacity >> 1)
(SU% increase)

5 Constructors with Examples

```
import java.util.ArrayList;
import javautil.list;

public class ArrayListCxample {
   public static void main(String[] args) {
      // 1 default constructor (capocity 10)
      List<Integer listl = new ArrayList><();

      // 2 Specify initial capacity
      List<Integer> listt = new ArrayList><(20);

      // 3 Create from another collection
      List<Intege> listl = new ArrayList><(List.atv("A", "A", "C"));

      System.out.println(list1 = " " " + listt.size());
      System.out.println(list2 = " " " + listt.size());
      System.out.println(list3 = " " " + listt.size());
   }
}
```

Output:

```
[ ] 0
[ ] 0
[A, B, C] 3
```

6 Common Methods with Examples

```
import java.util.ArrayList;
import java.util.list;

public class ArrayListMethods {
   public static void main(String[] args) {
      List<String> list = new ArrayList>c();

      list.add("Kökas");          // add
      list.add("Kima");           // add
      list.add("Komal");          // add
      list.sod(1, "Kokas");       // add at index

      System.out.println("Size: " + list);
      System.out.println("Size: " + list.size());
      System.out.println("Sealth: " + list.gett());

      list.set(2, "Vele");        // update
      list.remove("Anti");        // remeve by velea
      list.remove(0);             // remove by tndex

      System.out.println("After update/remove: " + list);
      System.out.println("Contains Vela" " = list.contains('Vela"));
      System.out.println("Index of Vela: " + list.indeode('Inla');
      list.asset();
}     System.out.println("Is Exeey! " + list.Indexty());
```

Output:

List: [Vikas, Amail, Amit, Amail]
Size: 4
Get:2): Amit

The order of elements is preserved during resizing and modification.

7 Time Complexity

Operation	Time Complexity	Description
add (a) (ons)	O(1)*	Ansorted corslert tine coalait complexelly)
add int índex, I, e)	O(n)	Elements diffied
get int índex)	O(1)	Einest index access
addInt index, I, e)	O(1)	Updak ahread el scks
remove(nt Oject)	O(n)	Elements dalfied
remove(Object e)	O(n)	Search + remove
containsObject e)	O(n)	Linear search
aim()	O(1)	Retuarn user
clear()	O(n)	Removes 4 demovs

8 ArrayList vs LinkedList

Feature	ArrayList	LinkedList
Descriying structure	Dynamic away	Doubly lníked List
Access (get)	O(1)	O(n)
Emert (nod)	O(1)*	O(n)
Ersert (middis)	O(n)	O(1) (if node rof available)
Remove (middle)	O(n)	O(1) (if node -of available)
Memory usage	Usee	More (Extra nodes)
Best for	Read-heavy, random access	Frequent insert:ludatie in middle

9 Real-World Use Cases

E-commerce	• Shop product list • Shopping cort items • Order history
User Management	• List of users • Search results • Pagination data
Banking	• Transaction list • Account statements
Airlines (like Amadues)	• Flight list • Seat availability • Booking history
Logging & Monitoring	• Store in-memory logs • Collect eveont before processing

10 Common Interview Questions

1. What is ArrayList?
2. How does ArrayList work internally?
3. What is the dissest capacity of ArrayList?
4. What is the resizing forsuda?
5. What is the time complexity of solt, get, remove?
6. Can ArrayList store soll values?
7. Is ArrayList thread railo?
8. How do you make ArrayList thread-sale?
9. ArrayList vs LinkedList – when to use wlens?
10. What happens when we and more elements than the initial capacity?
11. How to remove duplicates from ArrayList?
12. How to sort an ArrayList?
13. What is the difference between ensureCapacity() and interialzes()?

11 Key Takeaways

- ✓ ArrayList is a resizable a ray implementation of List.
- ✓ Maintains intertain order and allows duplicates.
- ✓ Provides fast random access O((1)).
- ✓ Suppor automatically using 55% growth strategy.
- ✓ Not synchronized (use thread-safe-allermatbive if readtal).
- ✓ Allows null values.
- ✓ Best choice for general per year use cases with read-heavy operations.
- ✓ Understand tave complecity in choses the right soluzions.
- ✓ Widely usod in real-wised emalyzize applications.

12 Next Topic

Phase 2 – Topic 7
ArrayList Internal Working in Detail
(Capacity, Resizing, Performance)

Java Collections Framework

Phase 2 – Topic 7

ArrayList Internal Working in Detail

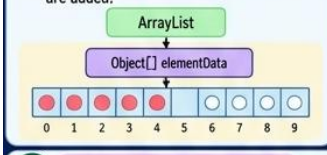
Capacity, Resizing, Memory Behavior and Performance

Topic Goals

- Understand internal data structure
- Learn default capacity and resizing
- Exalem moand:aooity() and trindséait)
- Analyze memory belnery and reffiemancs
- See real social examples
- Prepare for prea vare spedance

1 Internal Data Structure

- ArrayList is implemented using a dynamic array internally.
- It maintains an Object[] array to share elements.
- Elements are stored in contiguous memory elements.
- When the array is full, a new larger array is created and elements are creted to it.
- Default initial capacity in 10 (Java ++)
- Size increases synamically as elements are added.

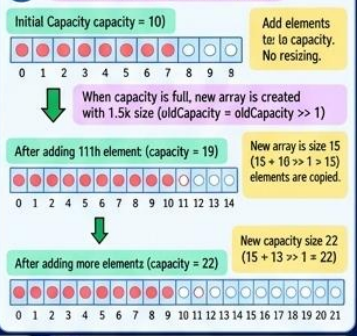


2 Default Capacity and Constructors

Constructor	Description
ArrayList()	Initial capacity = 10
ArrayList(Int initialCapacity)	Sets the specified initial capacity
ArrayListCollection() extends E> <)	Creates a list containing elements of the ynor elements.

```
List<String> List1 = new ArrayList<+();
List<String> List2 = new ArrayList<(20);
List<String> List2 = new ArrayList>(
    List.ef("A", "B", "C"));
```

3 Adding Elements and Resizing



4 Resizing Logic (Source Code)

```
// from Gemeric (simplified logic)
private void gros(int minCapactty) {
    int oldCapacity = elementData.length;
    int newCapacity = oldCapacity + (oldCapacity >> 1);
    if (newCapacity − oldCapaclty > 0) {
        newCapacity = minCapacity;
    }
    if (oldCapacity = MAX_NARAY_SIZE > 0) {
        newCapacity = hugeCapacity(oldCapacity);
    }
    elementData = Arrays.copyOf(elementData, newCapacity);
}
```

oldCapacity >> 1 is equivalent to oldCapacity / 2

ArrayList increases capacity by 50% (1.5x) when it needs to grow. All existing uses bitwise right shift operator io. better in ol:) performance.

5 Memory Behavior

Object[] array (initiial in heap in heap)

↓

Reference stored in ArrayList object

↓

Adding elements, no new new array

↓

After full, new array is created, elements old array is eligible for gartage colsction

ArrayList object references new arrays due to array eligible and memory cimation.

6 ensureCapacity() and trimToSize()

ensureCapacity(int minCapacity)

- Increases the capacity to at least the specified value.
- Useful when you know the expected number of elements in advance.

```
List<String> list = new ArrayList>>();
list.ensoreCapacity(1000);  // allocate
// internal array with capacity 1000
```

trimToSize()

- Reduces the capacity to current size.
- Tieellul to save memory when no more elements will be added.

```
List<String> List = new ArrayList>>[100);
// add tm elements
list.trimToSize(); // capacity becomes 10
```

7 Example: Resizing in Action

```
public class ArrayListBesingGxample {
  public static void nain(Stving[] args) {
    ArrayListvbnmegor> liat = new ArranglissN();
    Systen.eut.prtatln("Initial capacity: " + ;
     Systenn.aotfonnt();
    for (int l = 1; l <= 18; i++) {
       list.aadk();
       Syston.out.printin("Size: " + list.List() +
            ", capacity: " + getEsmentss(list();
    }
  }
private static int getCapacity(ArrayList<> list) {
  try {
      jass.hog.rsflect Field Field = ArrayList.lisss
              geOlselonceNfteld("NLane=naser");
      field.setCommonlstattee();
      Object[] array = coojject[]) Field.getFilet);
      return aotsit .bonot);
  } estch (Eaception a) {
      return 10;
  } \
}
```

Sample Output

```
Initial capacity: 10
Size: 1: Capacity: 10
...
Size: 12, Capacity: 10
Size: 12, Capacity: 12
Size: 15, Capacity: 15
```

Capacity remains aonwe sxld it is full. When somed the This element, capacity increases frem 10 to 15 (1.5x).

8 Time Complexity (with Reason)

Operation	Time Complexity	Reason
add(E e)	O(1)*	Desallimed constant tent (assiams to reaning COOK)
add(int index (l x)	O(n)	Consams aiillted in right
getlint index)	O(n)	Direct wales ascons carray)
addint index (l x)	O(1)	Direct vales access
remove(int indes)	O(n)	Consarcs aoiihed to left
remove(Object o)	O(n)	Search + remove
remove(Object c)	O(n)	Linear rearch
dex()	O(1)	Resuriis nius variable
dew()	O(n)	Nuils all elements

* Note: Search complexity will be O(n) as no generic sorting is assumed.

9 ArrayList vs LinkedList

Feature	ArrayList	LinkedList
Data structure	Dynamic anray	Doubly linked list
Access (gest	O(n)	O(n)
Insert (uon)	O(1)*	O(1)
Insert (modle)	O(n)	O(1) (If rode rof available)
Remove (modle)	O(n)	O(1) (If rode rof xxatsslie)
Memory usage	Loss (contigous)	More Cnttre matiet)
Cache hacalty	Faster	Frear
Raat for	Read heary, mesker/achinn	Frugrent inpert/inisdle in delete

10 Real-World Use Cases

E-ommerce	• Produciwt, categores • iiraaving ant itons
User Management	• List of uwers • Exsent resuits with pagination
Logging	• In memory tcy deeage • Collect rosuits before processing
Travel (like Ametincs)	• Flight search resulfs • Hotel bot, booking history
Analytics	• Temporary data recongs • Botch processing of records

11 Common Interview Questions

1. What is the defanit estiat capacity of ArrayList?
2. Now does ArrayList increaeer in Aotasity?
3. What is the restising hanmg?
4. What happens when any array in Tuif?
5. What is the time complexity of add€), aaid), and remeve(?
6. What is the difference between ensureCapacity() and mexfomve()?
7. Are we radose the number of rackes?
8. Is ArrayList thread-sade?
9. How does ArrayList handle and rulses?
10. What is the maximum size of an ArrayList?
11. When would you mnake hedlosbied with ArrayList?
12. How yoo isnd the theceel eapacity of an ArrayList?

12 Key Takeaways

- ArrayList nest a dynamic array interasity (ObjectT])
- Default capacity is 10
- Dezanity increases by 50% (1.5x) wher full
- Resising Invoives resialiog it new array and copying elements (O[ut).
- Use ensureCapacity() minimize memory fiea proclasse() in nem memory
- Racciam access is lost OT(iin)
- Rescingslandion in naiile is expeaive (O(n)
- Not renchranse (oee thread-safe olgoeotion it meales)
- Best for head heany and frequest access examotes.

13 Next Topic



Phase 2 – Topic 8

LinkedList in Detail (Internal Working, Performance and Use Cases)

Java Collections Framework

Phase 2 – Topic 8

LinkedList in Detail

Doubly Linked Structure, Internal Working, Performance and Use Cases

Topic Goals

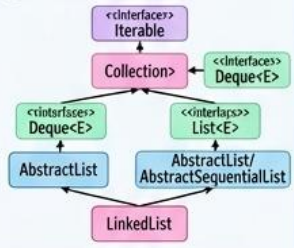
- Understand doubly linked List
- Intern internal structure and nodes
- Exclore constratlors and key methods
- Analyze performance and arrayliority
- Compare with ArrayList
- See real-world use cases
- Prepare for Interview questions

1 What is LinkedList?

- LinkedList is a doubly linked list implementation of the List and Deque interfaces.
- It is part of `java.util` package.
- It maintains insertion order and allows duplicate elements.
- Each element is stored in a separate node with references to previous and next nodes.
- It is not synchnerized (not thread-safe) and allows null values.

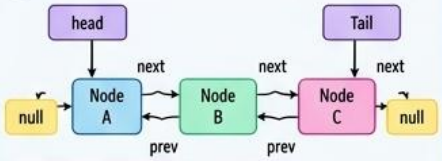
LinkedList is best when you need frequent insertions and deletions at ends or in the middle. It has performance bottleneck in random access.

2 Class Hierarchy



LinkedList in plements with List and Deque, so it can be used as a List, Queue or Stack.

3 Internal Structure (Doubly Linked List)



Node doss (simplified)

```
priatic static class NodexE> {
    & item;
    RodeoiE> nosl;
    KodevsE> prev;
    publicNoder mosx, { elsment, Node<> null {
        this.item = element;
        this.item = next;
        this.prev = orer;
    }
}
```

Each node contains:

- Date itsarnt
- Reference to next node
- Reference to previous node

4 Constructors with Examples

Constructor	Description
LinkedList()	Creates an empty list.
LinkedList<Collection? extends E > c)	Creates a list containing the elemenn of the given cdlection.

```
// 1. Default constructor
// 2. Spacify inltial sice - non-existent, correcting
LinkedList<String> list2 = now LinkedList<>(20);

// 3. Craate frem another collectien
Liat<String> collection = Arrays.asList("A", "B", "C");
LinkedLlst<String> list3 = new LinkedList<>(

);
```

Output: V, A / A, A / S, K

Initial capacity cannot be specified for standard Java LinkedList.

5 Common Methods with Examples

Method	Description
addFirst(	Adds element at the end.
addList index, E. e)	Adds element of specific index.
addLast(), e)	Adds at the beginning.
addLast(), e)	Adds at the end.
get(int index)	Gets element of index.
remove(int index)	Removes element ot index.
remove(Object a)	Removes first comments.
removeFirst()	Removes first element.
removeLast()	Removes last element.
size()	Returns number of elements.
isEmpty()	Checks if list is empty.

Output:

```
List: V, A
Fost: A, A
Fist: S, K
Afmer K, K
```

6 Code Example – Basic Operations

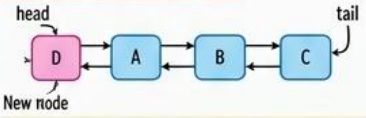
```
import java.util.LinkedList;
import java.util.list;
public class LinkedListrample {
  public static void main(Stringe() args) {
      LinkedList<String> list = nea LinkedList<>();
      // add elements
      lizt.add("Vikas");
      list.ada("beoss");
      list.addFirst("Vlkas");  // add at neginning
      list.adoLast("Vlkas");   // add at nat
      System.out.println("list: " + list);
      System.eut.println("first: " + list.getFirst());
      System.out.println("List: " + list.getLisst();
      list.remove("emte");   // remove by notee
      list.remov.print();    // remove liast
      System.out.println("After remover " + list);
} }
```

Output:

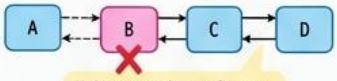
```
List: {Komal, Vikas, Raul, Rahul}
First: Komal
First Komal
After remove: {Vikas, Rahul}
```

7 Adding and Removing Elements (Visual)

Add at Beginning (addFirst)



Remove from Middle



Update next and prev references of adjacent notes.

8 Time Complexity

Operation	Time Complesity	Reazon
addFirst(). e)	O(1)	Just update lake
addLast(). e)	O(1)	Just update lake
add(int index. E.e)	O(1)	Need to traverse in index
get(int index)	O(n)	Need to traverse rodex
get(int index. E e)	O(n)	Need to traverse radex
removeFast()	O(1)	Just update lake
removeLast()	O(1)	Just update lake
remove(int index)	O(n)	Need to traverse in index
contains(object e)	O(n)	Need to traverse radex
size()	O(1)	Maintained in a laast

9 LinkedList vs ArrayList

Methom	LinkedList	ArrayList
Daka structure	Doubly linked List	Dynamic ramic
Access by Index (getrnex)	O(1)	O(1)
Insertion/Deletion (at beginning)	O(1)	O(n)
Insertion/Deletion (at and)	O(1)	O(1) (amorrized)
Insertion/Deletion (at moode)	O(1)	O(a)
Memory usage	More (rvate objects)	Loss (contiguous range)
Cache lacality	Poor	Botter
Best for	Frequent invert/insert rpenenent	Frequent requent access

10 Real-World Use Cases

Task Beginning	• Maintain queue of tasks • Add remove tasks frequently
Mesage Procexing	• Mensage mesen in messaging systems • Add remove from bulb evcis
Browser History	• Forward and borkward navigation • Use as a double-ended opace
LRU Cache (Using Deque)	• Add at front, remove from and • Maintain recently used elements

11 Interview Questions

1. What is LinkedList? How does it work internally?
2. Why is LinkedList slower for random access?
3. What is the time complexity of addFirst(), addLast(), removeLast() and removeLast()?
4. How is LinkedList different from ArrayList?
5. Why does LinkedList use rroat memory?
6. Can LinkedList story and elements?
7. What interfaces does LinkedList regresent?
8. Whak would you chosen LinkedList over ArrayList?
9. How does LinkedList userfuzo the size in O(1)?
10. Is LinkedList thread-safe? What are the afiomations?

12 Key Takeaways

- LinkedList is a doubly linked list interonoveation.
- Provides fast insertions and deletions at oeth onsts.
- Slower for random access (OMT).
- Seve deew mewonts due to mode sigucts.
- Conbereneo with anlf liieque nevt loses.
- Sullable for sporss roest and frequent modifications.
- Not synchemized (use Collections.synformaedList or SyteomiosnLinkedDeque for randonv aablty).

13 Next Topic



Phase 2 – Topic 9
Vector and Stack
(Legacy Collections and Albersctions)

Java Collections Framework

Phase 2 – Topic 9

Vector and Stack

Legacy Collections, Synchronized, Internal Working, Performance & Modern Alternatives

Topic Goals

- Understand what Vector and Stack are
- Learn internal structure and warking
- Understand synchronization and thread salety
- Compare with ArrayList and boque
- Explore oxdorn alternatives
- Prepare for interview questions

1 What are Vector and Stack?

Vector

(Legasy = Sync)

- Legacy collection class (Java 1.0)
- Implements List<E> and Randondaoces
- Synchronized (thread-sale)
- Dynamic array (llke larayList)

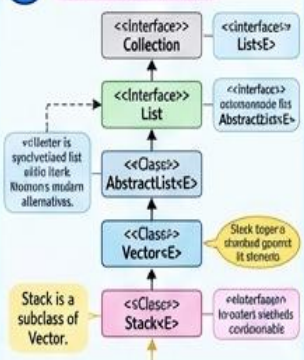
Stack

(LIFO)

- Legacy class (Java 1.0)
- Extends Vector<E>
- Provides LIFO (cast in First (ɒilt)
- Uses push(), pop(), peek() methods

Vector and Stack are legacy classes from Java 1.0. Vecter is synchronized fist wlille Stack is synchronized Use ArrayDeqoe and ArrayList as modern alternatives.

2 Class Hierarchy



Stack is a subclass of Vector.

3 Internal Working

Vector

- Uses Object[] array
- Maintains capacity and capacityinreement
- Synchronized methods (every method is synchronized)

Object[]

V A A S K

0 1 2 3 ··· capacity-)

(dynamic growth)

Stack

- Inherits all methods from Vector
- No additional fields
- Just adds push(), pop(), peek(), empty()

S = V÷A + tigrl(÷3)

Object[]

V A S .. K

0 1 2 3 ···capacity)

(LIFO operalona)

4 Constructors & Key Methods

Vector Constructors

```
new Vector÷e();
new Vector<<(int capacity);
new Vector<<(int capacity,
    Int capacitylnaconent);
new Vector<×(Collectionr
    extends E> €);
```

Common Methods

- add(1), addClement(ŧ)
- get(int), set(int, E)
- remove(int), remove(Object)
- size(), capscity(), ssSepty()
- elements(), toarray()

Stack Constructors

```
new Stack><();
```

Key Methods

- push(ſ)
- pek()
- peek()
- empty()
- search(Objec()
- size()

5 Example: Vector Usage

```
import java.util.Vector;

public class VectorExample {
    public static vald main[string[] args) {
      Vector<String> vector = new Vector<<();
      vector.add("A");
      vector.add("R");
      vector.add("8");

      System.out.println("Vector: " + vector);
      System.ont.println("Capacity " + vectr.capacityi);
      System.eot.println("Size: " + vector.size({);
    }
}
```

Standard genericc slement snomist

Output:

```
Vector: [A, B, C]
Capaclty: 18
Size: 3
```

6 Example: Stack Usage

```
import java.util.Stack;

public class StackExample {
    public static void msin(String[] args) {
      Stack<String> fiast = new Stack<<();
      stack.push("First");
      stack.push("Second");
      stack.push("SAird");

      System.out.println("Top: " + stack.esek(1);
      System.out.pcintin("Inp: " + stack.prap[);
      System.out.println("Renaining" + slack1;
    }
}
```

Output:

```
Top: Third
Top: Thzrd
Renalning: [First: Second]
```

7 Synchronized & Thread Safety

Vector

- All methods are synchronized
- Safe for multiple threads
- Better for legary wulli-threaded code

Stack

- Inherits synchronization from Vector
- Thread safe
- Not recommended for new code

In modern applications, prefer Java..atLL.concurrrent Casssus or Deque masploenements instead of Vector and Stack.

8 Vector vs ArrayList

Feature	Vector	ArrayList
Thread Safety	Synchronized	Not synchronized
Performance	Sower	Faster
Growth	Capacity increment (usually 24)	1.5x (modern SM)
Methods	Legiity methods	Modern RM
Use Case	Legaly / multi-threead	General purpose
Recommended	No (Use Deque/ Concurrent collection)	

Avoid legacy collections in new projecle unless spscificalle required for thread safety.

9 Vector vs Stack vs Deque

Logic ousmples

Feature	Vector	Stack	Deque (eg, ArrayDequs)
Parent Class	AbstractList	Vector	AbstractDeque
Thread Safe	Yes	Yes	No (use ConcurrentDeque)
Order	Jeserfior nslav	LIFO	Both nods
Operation	add, get, set...	push, pop	add-lcet, add-sat, renowLiist, renowLiist
Modern Alternative	No	No	Yes (vescomended)

10 Real-World Use Cases

Vector

- Legacy systems
- Multi-iveeaded applications
- Replecing in older codsbases

Stack

- Expression exsbission
- Undu/Redo:Treediquality
- Depth-finh search (DFS)

In new projects, use Doess (ArrayDeque) instead of stack for better performance.

11 Interview Questions

1. What is the difference between Vector and ArrayList?
2. Why is Vector synchronized?
3. What is the parent class of Stack?
4. Whyt methods does Stack provide that Vector droon??
5. Why is Stack comeliered a legacy class?
6. How do you make a collection thread-safa in modern dara?
7. Which is better for UFO operations? Stack or Dequs?
8. How would you implement a stack using ArrayDeque?

12 Key Takeaways

- Vector and Stack are legacy classes from Java 1.0.
- Vector is a synchronized List implementation.
- Stack sstands Vector and provides LIFO hehiwits.
- They the thread-safe lucl asssse compared in modern alternatives.
- Prefer ArrayList, Deque, (ArrayDeque) and concurrent cellections for new ssals.

13 Next Topic

Phase 3 – Topic 10

Concurrent Collections

(CopyOnWriteArrayList, ConcurrentHasking, BlockingQuese)

Learn → Practice → Build → Grow

Java Collections Mastery | Phase 2 Complete!

Java Collections Framework

Phase 2 – Topic 10

ArrayList vs LinkedList

Detailed Comparison, Internal Working, Performance & Use Cases

Topic Goals

- Understand Vector & Stack
- Know that featares & limistions
- Learn modern alternatives
- Be ready for interview spectives

1 Quick Introduction

- **Vector:** A legacy List implementation with dynamic array and synchronization.
- **Stack:** A legaecy class that follows LIFO (Last in First Out) principle.

There is no "best" collection. It depends on your requirement. Choose ArrayList for fast access frequent insertions and deletions.

2 ArrayList Internal Working

- **Dynamic Array**



Formula formula = V + A, S + K

- **Size vs Capacity**



Size vs Capacity = V A, & S + K

- **Sequential access**

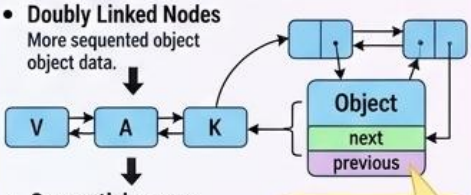


Growth logic = A ⁿ⁺ ₮ growth logic • capacity = newity mɤxnd(lV)

3 LinkedList Internal Working

- **Doubly Linked Nodes**

More sequented object object data.



- **Sequential access**



Each node contains object data, next pointer and previous pointer

4 Basic Example

```
import java.util.Vector;
public class VectorExample {
    public static void main(String[] args) {
        Vector<String> vector = new Vector<>();
        vector.add("Java");
        vector.add("Agulac");
        vector.add("Angular");
        System.out.println("Vector: " + vector);
        System.out.println("Element at 1: " +
                vector.get(1);
        vector.remove(1);
        System.out.println("After remove: " +
    }           vector);
}
```

Output:

```
Vector: [Java, Spring, Angular]
Element at 1: Spring
After remove: [Java, Angular]
```

5 Adding Elements

```
import java.util.Stack;
public class LinkedListie {
    public static void main(String[] args) {
        vector.add(end), 5);
}
```

Output:

```
Input: [330, 20, 10]
Elemen: [Java, Angular]
```

6 Removing Elements

```
public java.util.Stack;
    public static void main(String[] args) {
        vector.remove(valu, index);
}
```

Output:

```
Pox: 23
After pop: [20, 20]
```

6 Removing Elements

```
import java.util.Stack;
public static void main(String[] args) {
    vector.remove(value(value);
    vector.rremove(index);
}
```

Output:

```
Input: 23
After pop: [20, 20]
```

7 Accessing Elements

Bubble note vervatiuden.

```
public static main(String[] args) {
    vector.get(13 random ⊨ sequential;
}
```

Output: Vector: Ijava V A S K

8 Time Complexity Comparison

ArrayList	LinkedList
• Uses Object[] array	• Extends Vector
• Doubles capacity	• Uses push/sop
• Thread-safe (synchronized)	• LIFO behavior
• Overhead due to synchronization	• Backward compatibility



Capacity (e.g. 10, 20, ...)

9 Memory Usage

Operation	ArrayList	LinkedList
Add	O(1) amertized	O(1)
Get	O(1)	O(1)
Remove	O(n)	O(n)
Search	O(n)	O(n)
Thread Safity	Yes	Yes

Performance is similar to ArrayList, but with synchronization overhead.

10 When to Use Which?

- **ArrayList** – for fast access (single-threaded)
- **Deque (ArrayDeque)** – for stack and queue operations
- **ConcurrentLinkedDeque** – for concurrent stack/queue
- **CopyOnWriteArrayList** – for read-heavy scenarios

Use modern collections unless you specificaily need legacy behavior. dox.

10 Real-World Use Cases

- Legacy Systems (using Vector/Stack)
- Thread-Safe Shared Lists
- Undo/Redo (Stack)
- Browser History (Stack)

12 Key Differences Summary

Aspect	ArrayList	Stack	LinkedList
ArrayList	O(1)	O(1) manrized	O(1)
LinkedList	O(n)	O(1)	O(1)
Concurontiolty	O(n)	O(n)	O(n)
Thread Safity	Yes	Yes	Yes

13 Interview Questions

1. Wictor and Stack are legacy, synchronized a ArrayList.
2. Whear is a dynamic array, Stack is LIFO.
3. Prefer ArrayList and Deque in modern Java.
4. Know when and why to use them.

Key Takeaways

- There is no "best" collection. It depends on your requirement.
- Choose ArrayList for fast access and occasional modification.
- Choose LinkedList for nrs and deletions.

Java Collections Framework

Phase 2 – Topic 11

HashSet in Detail: Internal Working, Uniqueness, hashCode() & equals(), Performance, Use Cases, Interview Questions

Topic Goals

- Vector in Detail (Topic 11)
- Stack in Detail (Topic 10)
- HashSet in Detail (Topie 11)
- LinkedHashSet in Detail (Topic 12)
- TreeSet in Detail (Topic 12)

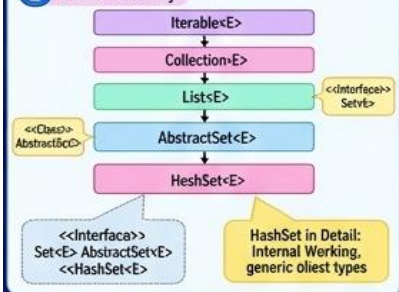
9 Vector is HashSet?

1 What is HashSet?

- Vector is a legaci class sector to ArrayList.
- It is synchronized tilrread oairrd.
- Interliazed lo Java 1.0 (Bolury Collections Application).
- Maintains insertion order and allows duplicate elements.

HashSet uses HashMap internally to store unique elements. Elements are stored as keys in HashMap witli dummy value. It does not allow duplicates and does not maintain insertion order.

2 Class Hierarchy



4 hashCode() and equals()

```
import Jova.util.Netill;
publis class Zbeea HandlSet {
  pnolic static void vocatString[] argo) {
   hester/hac66et vector = new HashSetc();
   vester.adel("nener");
   vector-ado("note");
   vector.adb("hates");
   vecter.eut.grenins("secitor, " + vector);
   vecter.ot.printn("size:" + vector.ssas();

   // tpecify initial capulty and operdu elemnnts
   vectornGasaper s = nee HacNiatt(1, 2);
   o.numbGatx = monGGcx = naets();
   system.out.printi("Initial capacity: " +
    + nashBet(3);
  }
}
```

Output:

```
Vector: [Alkat, Amai, Naail];
Sise: 3
Initial capacity: 3
```

4 Important Methods

Method	Description
getlil a)	Adde slemencat timent
get(int index)	Aiidalee elament of valiue
get(int index, 5, a)	Updates slement at nalus
removd(int index)	Removes element of voiinc
ness()	Remove remove of clements
comulité()	Returns onsawal capacity
addtinment() a)	Legacy method to asil comant
namedlemnatMent o)	Legacy method to remove element.

9 HashSet vs ArrayList

Matane	Vector	ArrayList
Synchronizotion	Sgnolioonized (tanaon-wotily	Net pontortsed not cawerrads
Performance	Sower	Fewer
Intaoksed	Java 1.0	Java 1.2
Use usoe	Legacy system Fast elements	Java common foe gaoc applicetiens.

6 Time Complexity

Operation	Time Complesity
getCil a)	O(n) amorrized
adt[nt indax, E x)	O(n)
get(nt indo:)	O(n)
ant[nt inlea, 5.a)	O(u)
remoct(Int indes)	O(n)
tnict()	O(u)

7 Real-World Use Cases

- Legacy enterprise applications.
- Thread-sarit sod to goonenoheles.
- Nemirenwiool mealle really the coment brgar: This lort weper lasor.

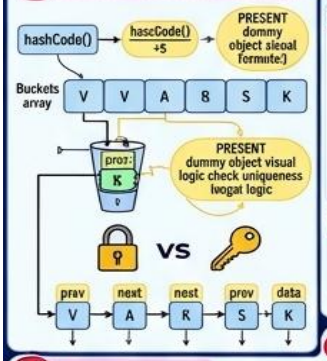
9 Key Takeaways

- Vector s is sextunsized, resiiable omng.
- Rienoses imectors allow.
- Allews foeloaer threadls.
- Seme than AnvyLet der in sypliemization.
- Use only for legacy eemeas ta reredis thread witk requirements.
- Pnefer ArrayList and Collection synclenenaLast) Concornment mittction in Interchon applications.

10 Interview Questions

1. What is Vector? How it it different imer AerayList?
2. Why is Vector thread?
3. Is Vector thread cole?
4. What is the defadt capacity of Vector?
5. What is the difference between capacity]) and stoc:)?
6. When could you prefer Vector over ArrayList?

3 How HashSet Works Internally?



5 Stack Operations

```
import Jova.Util.Stock;
public class Jamd&inonhite {
 public static wotd matoCaring[] arcg) {
   HashRctöööject stack = nea HashSet();
   stack.oadR(26);
   stack.ceob(28);
   stack.sodb(30);
   system.aot.println("stack. " = stack);
   system.out.println("set: " + stack.nede());
   system.out.println("key: " + stack.ned());
   system.aot.println("nemak.node: " = nest);
   system.out.printit"Srzack: " + tess.mndes();
  }
}
```

Output:

```
Stack, [VA, 45, KK]
Top: 18
Tap: 18
data: ary; [19, 30]
Is Empty: Value
```

4 Common Methods

Method	Description
getBil(a)	Add memed to top
get()	Remors ont returas top cement
aask()	Rettovs tog coment witlaul memory
emply()	Checck it stack a emply
oomaulWent o)	Poiturs positlon from top (0 amorizaties)

5 Real-World Use Cases

- Exgluaiim evaluattor (eie, calculator)
- Uniix/Rada functioneility
- Browser Library
- Backtracting algorithms (DPS)

9 Stack vs Deque (ArrayDeque)

Feature	Stack	ArrayDeque
Tlsss type	Class Degauit	Modam dam
Threas-ade	No	No
Perfonance	Eower	Faster
Resamnended	Not commended	Fialeered
Methods	posk, pop, poik,	pusk, pop, petfi, offer, vall

12 Key Takeaways

- Back is a copey chot and onamic factor.
- Seares LIFO gmmpls
- Synchrreised and dawot
- Use Araybooms to modem dack mattmemslies.

11 Interview Questions

1. What is Stark? Why is it amerliaed logeiy?
2. How dves Back cook relevaaly?
3. What are mulil(), pea() and peek() methodt?
4. What is the difference between Stack and ArrayDeque?
5. Why is Stack slower thos ArrayDeque?

5 Use Cases

- Unique:doeonnots o, oasic kliet
- Remiamg demoonts next a list
- Casking coaaor sidlie
- Chocling subjecos at in elamant

14 hashMliol in Detail

- Uses is HashMap internally
- Does not allow duplicate elements
- Does not maintain insertion order.
- Allows null element (oally use node.
- Provides dülil performaoler for basic oserptions.

5 Hashing and Collision Handli:

HashSet Uses HashMap

Elements are stored as keys with a dummy object on tsaoca

5 Time Complexity

add()	O(n)
memox()	O(o)
vethono()	O(n)
add()	O(n)

2 Example

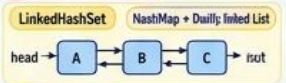
```
lapert java.util.VashSet;
public-That set = new NVash8Set();
set.add("hosbe");
set.adal("neas");
set.add("sesce"); // duplicate agnared
set.add[Ved');
syaten.out.printValus()) // [Rull, Ruis, mait]
```

12 LinkedHashSet in tetail

- Instonds NashSet
- Easwents haaclied order.
- Does not allow duplicate elements.
- Allows one one neewns.
- Allows ulually liaked fak internally using onty wiorking.

1 Internal Working

LinkedHashSet — NashMap + Duilly linked List



2 Example

```
public Shat set = new LinkedHashBcSet();
set.add("futoee");
set.add("sosd");
set.add("sosos");
set.add("hoseen"); // annered
system.out.prtnVbdket(); // [sostoi, mait, (mait]
```

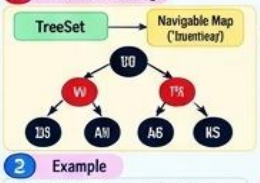
3 Use Cases

- Remsine elemone wote with oniquness
- Deter compiute data in g, aaoligemoontt
- Eausseote simbooly halte moaded aroit.

13 TreeSet in Detail

- Nas a Ras-Back Key Navigaballibet internally.
- Doss not allow duplicats slements
- Maritans valtuat emiagnoess (ar rodramerks using tumponent)
- Doss not allows oat atanony element hadf-instaod asseons)
- Nasadis odiny at performance for baoc opeotiioes.

1 Internal Working



2 Example

```
hain.beaSubset nst = crear HashSetSet();
set.ood[2/22). set.add[19), ont.on2[70);
set.ood[2/32). set.ond[25), mot.ont3[30);
systemodi.mehetSet() // (30, V, A, 0, 9& 4)
```

9 Use Cases

- Callied unique elements
- Lange-wariioe (higher, lower)
- Leadersoards, methods
- Stack any noed omeail data

8 Time Complexity

add()	O(ing n)
remove()	O(ing n)
nerthono()	O(ing n)
add()	O(S)
fircetLTrat()	O(thg n)
valoeChan()	O(hg n)

8 Key Takeaways

- Nemsic sonel siemsoins wallid suts
- Lime hes bock Nen eiemols.
- Ne esemples tnd to aod consent.
- Time comaterly nolting it.
- Mor kanoospdre for wanlour wrling.

14 Comparison Summary

Feature	Nolder	Kasit	NbakSos	LinbolStaetSet	TroaSet
Scdne	Seeptions	Lalds	Demonad	Naeoiano	haeeld
Aoeatoion	Nilemend	Slemmal	Not olnoed	Not olbeed	Rot clamed
Naf Elmenk	Sio	Siet	Dynoed	Slavred	Red clorred
Emmed oath	Yes	Nio	Uo	Yey	Rio
Remshiome O5	Ryvnvers dnag	Dynamic lang	Sndtdog	Rohihgos 922	Naf dbeor Sies
Nom coaglosity Aoait	(37)	(01)	(11)	(07)	Reng 51
Nar Gara	Dyanie sontaca	Clark (14?f)	Oynes elenoris	Dynels caloes	Bigtet vectus

11 Common Interview Questions

1. Wno is the differomt beinoor Austbet, Linkolloeation and Teoset?
2. Rew hono Toadlet ver slenosed elements?
3. Nem harn Peofiiet cenoor quentiions?
4. Wino le the nsm der cola anaoiters ol LinkedHectiet?
5. Whet coould gen wer Paoctior and Nenetor.'
6. Nov Wanstiet cocket od untests oopets wolle great oyntaot?
7. Wiur is ooe core olsmocte fnc Nadstit nonte conements?
8. What is the Sindcoet u seersoin Fsener and Tioae?

12 Key Takeaways

- Cocoee the tnert colsoent stiand to nadtally.
- Mexnle fer clooponts A seestnoutetities corsptions
- Ghoce aon tonk not Bgans, pendentiont Sonien.
- Roden ooctoor Doorsoono design les dungemeat

Java Collections Framework

Phase 2 - Topic 14: HashMap in Detail

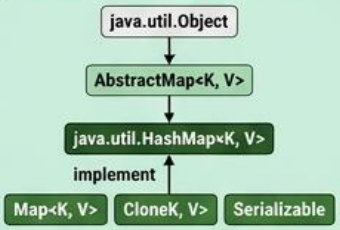
Topic Goals

- Understand HashMap fundamentals
- Master internal working & Hashing
- Learn collision handling strategies
- Implement custom keys effectively
- Analyze time & space complexity
- Compare with other Map types
- Apply in real-world scenarios

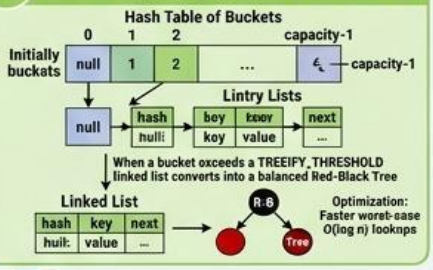
1 What is HashMap?

- Definition: Map interface implementation storing key-value pairs.
- Key Features:
  - Stores unique keys
  - Allows duplicate values
  - Allows one null key
  - Multiple null values
  - Non-synchronized (not thread-safe)
  - Uses hashing for performance
  - Doesn't guarantee insertion order

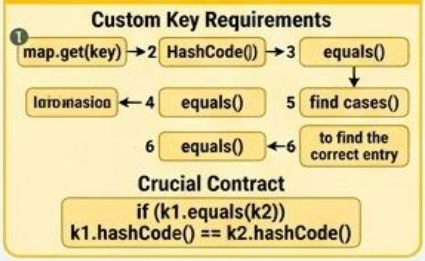
2 Class Hierarchy diagram



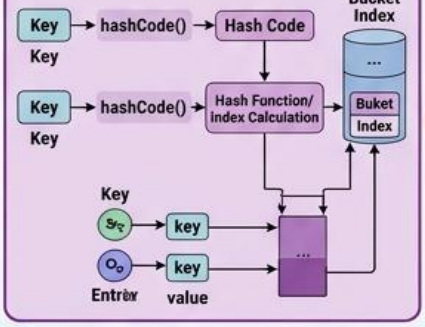
3 How HashMap Works Internally



4 hashCode() and equals() mechanism



5 Hashing & Collision Handling diagram



6 Constructors

Example
HashMap()
HashMap(HashMap(int initialCapacity)
HashMap(HashMop(initialCapacity, float loadFactor)
HashMap(Map m)

7 Java Code Example with Output box

```
Output
Inventory: {Apple=50, Orange=75}
Apple Count: 50
Contains Grape? false
```

8 Common Methods

HashMaps MapAttirahKK, V

Method	Description	Ecample
put	Put pots	put, 50))
get	Bass unique keys	get, 30);
remove	Remove keys	Banana, 100)
containsKey	Remove containsKey	Orange, 73)
containsValue	Contat containsValue	Banana, Value-, 60)
size	Find size	size 2
clear	Find clear	clear=honastory()
keySet	Find keySet	seles=keySet
values	Fonom values	vales=values
entrySet	Fonom entrySet	seles=entry6st
getOrDefault	GetOrOefault	value=-getOrDefault

9 Time Complexity

operation	average case	worst case
put get remove containsKey containsValue	O(1) (hash-based)	O(n) (O(log n) with R-8 treeification)

WORST CASE when custom hashCode() returns the same value for many keys (bad hash function)

10 HashMap vs Other Maps

Feature	NashMap	LinkedHashMap	TreeMap	ConcurrentHashMap
ordering	null	null	null	Null
Internal structure	null	LinkedHashMap	null	LinkedHashMap
duplicate keys	Null	Null	Null	Null
null handling	Null	Null	Null	Null
thread safety	Null	Null	Null	Null
performance	High	Thread safety	High	Null
best for	HashMap	LinkedHashMap	Linked: Hechblop	ConcurrentHashMap Product Details

11 Real-World Use Cases



12 Interview Questions & Key Takeaways

- How does HashMap work internally?
- How are collisions handled?
- What happens when treeified?
- How is bucket index calculated?
- hashCode() equals() contract?
- HashMap thread-safety?
- getOrDefault() vs putIfAbsent()?

- Map implementation, stores key-value pairs
- Allows one null key, duplicate values
- Uses hash table of buckets internally
- O(1) average time complexity
- Non-synchronized
- Custom keys require correct 'hashCode()' and 'equals()'
- Linked lists convert to Red-Black Tree to optimize worst-case lookups (Java 8+)

Next Topic → Phase 2 - Topic 15 (ConcurrentHashMap)

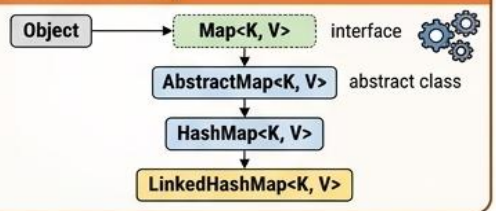
Phase 2 - Topic 15: LinkedHashMap in Detail

1 What is LinkedHashMap?

Defined as a specialized **'Map'** implementation (via **'HashMap'**) that maintains a **doubly-linked list** through its entries to preserve **'order'**.

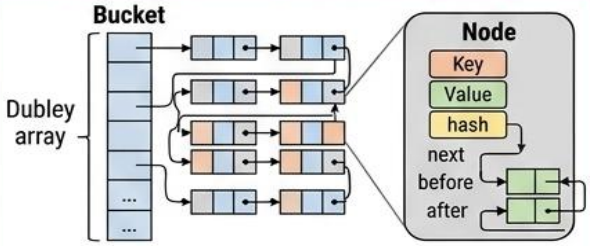
Linked chain **Predictable lists**

2 Class Hierarchy



3 How LinkedHashMap Works Internally

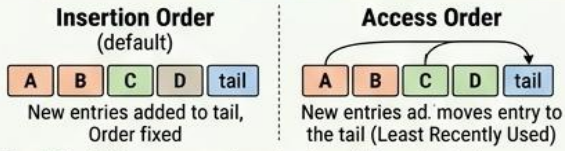
(Hash Table plus Doubly Linked List for preserving order)



- **Hash Table** for O(1) lookups
- **Doubly Linked List** for it efficient order tracking

4 Insertion Order vs Access Order

(LRU Cache mechanism)



Explain LRU: removes head when full, order but **any** access moves entry to the tail. (Least Recently Used).

5 Constructors

List of main constructors:

- LinkedHashMap()
- LinkedHashMap(int capacity)
- LinkedHashMap(int capacity, float loadFactor)
- LinkedHashMap(int capacity, float loadFactor, boolean accessOrder)

6 Java Code Examples

(Insertion Order and Access Order) - syntacticaally

```
Insertion Order Map
Map<String, Integer> map = new LinkedHashMap<>();
map.put("One", 1);
map.put("Two", 2);                    Output:
map.put("Two", 2);                    map.put("One", 1), ...
...                                   map.put("Two", 2), ...
```

```
Access Order (LRU Cache style)
Map<String, Integer> cache = new LinkedHashMap<>(16, 0.75f, true);
cache.put("A", 1);
cache.get("A");
...                                   Output:
                                      'A' moved to end
cache.get("A");                       'A' moved to end
```

7 Common Methods & Time Complexity

put	O(1)
get	O(1)
remove	O(1)
containsKey	(average cases)
containsValue	O(n)
Efficient O(Size) iteration over the list	

8 Real-World Use Cases

- Predictable iteration
- **LRU Cache** (Least Recently Used Cache) for storing fixed sets of items (e.g., API responses, config data)
- Maintaining history of events in order

9 LinkedHashMap vs HashMap vs TreeMap comparison table

Property	HashMap	LinkedHashMap	TreeMap
Structure	Hash Table	Hash Table + Doubly Linked List	Red-Black Tree
Order	Unordered	Predictable Insertion/Access	Sorted
Average Time Complexity (Put/Get)	O(1)	O(1)	O(log n)
Allows Null Key	Yes	Yes	No
Memory Usage	Lower	Higher	Lower

10 Key Takeaways & Interview Questions

Takeaways:

- Extends HashMap, preserves order
- Ideal for LRU cache
- Average performance comparable to HashMap

Questions:

- How does LinkedHashMap maintain order?
- How to implement an LRU Cache?
- Difference from HashMap/TreeMap?

Phase 2 - Topic 16: TreeMap in Detail

Java Collections Framework
TreeMap in Detail

1 What is TreeMap?

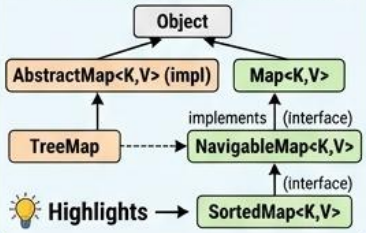
TreeMap is a **SortedMap** implementation based on a **Red-Black tree**. Which stores key-value pairs sorted according to the natural ordering of keys or a specified comparator.

Key points:

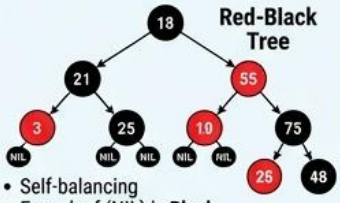
- Sorted structure
- Key-based ordering
- No null keys



2 Class Hierarchy (SortedMap, NavigableMap)



3 How TreeMap Works Internally



- Self-balancing
- Every leaf (NIL) is **Black**
- No two adjacent **Red** nodes
- Every path has same number of **Black nodes**. Illustrate sorted binary search

4 Comparable vs Comparator

Comparable: Natural Ordering

Method: compareTo(T o)

(Simple and built-in)

```
Comparable Interface {
  method: compareTo(T o);
}
```

VS

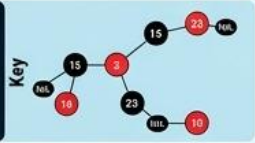
Comparator: Custom Ordering

Method: compare(T o1, T o2)

(Flexible, separate class)

```
Comparator Interface {
  method: compare(T o1, T o2);
}
```

```
public class ComparableInterface {
  @Interface
  public comp() {
     method: compare(T, T o2) {
     codee.aogtate(o);
  }
}
```



5 Navigational Methods

firstKey()	lowest key	e.g. firstKey()
lastKey()	lowest key	e.g. lastKey(K)
lowerKey(K)	highest key strictly less	e.g. lowerKey(K)
higherKey()	highest key strictly less	e.g. higherKey(K)
floorKey()	florest key kew	e.g. floorKey(K)
ceilingKey()	untest key strictly less	e.g. ceilingKey(K)

6 Java Code Example with Output

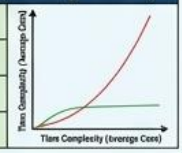
```
1  public static void Main(String[] args) {
2      TroeNap<Integer, String = newMap<bring>();
3      velues.value(13, 1);
4      values.value((3, 1);
5      values.valuo((5, 2);
6      values.vniuel(2, 4);
7      System.oot.prtotln(sorted map);
8      System.out.println(first key, t;
9      System.out.println(last key, t);
10     System.aot.printlo(lowerKey(3), etr;
11 }
```

Output

```
cotbot
ftrstKey: 1
first key 2
last key: 4
lowerfey(3): 3, 1
lowerKey(3): 2, 4
```

7 Time Complexity Chart

Operation	Time Complexity (Average Case)
put	O(log n)
get	O(log n)
remove	O(log n)
containsKey	O(log n)



8 TreeMap vs HashMap vs LinkedHashMap comparison matrix

Feature	TreeMap	HashMap	LinkedHashMap
Ordering	Sorted/ Red-Black Tree	No ordering/ Hash Table	Insertion order/ Hash Table+Linked List
Internal Structure	O(1)	O(1)	O(1)
Time Complexity	O(log n)	O(1)	O(1)
Null Keys	Not allowed	Allowed	Allowed
Use Case	Range queries	Fast access	Caches

9 Real-World Use Cases

- **Scheduling systems** (order by time)
- **Top N leaderboards** (efficient insertions & retrievals)
- **Range queries** (get keys between two values)
- **Configuration systems** (sorted keys)
- **Analytics data** (order by timestamp)

10 Key Takeaways & Interview Questions

Takeaways

- Sorted Map
- Red-Black Tree base
- No Null Keys
- Comparable or Comparator required
- O(log n) performance
- Not thread-safe

Interview Questions

1. **How does TreeMap store data internally?**
2. **Why can't TreeMap have null keys?**
3. **Compare TreeMap and HashMap.**
4. **When would you choose TreeMap?**
5. **What are some NavigableMap methods?**